成绩	

上海大学 2014～2015 学年度秋季学期试卷 (A 卷)

课程名：面向对象程序设计 课程号：08305121 学分： 5

应试人声明：

我保证遵守《上海大学学生手册》中的《上海大学考场规则》，如有考试违纪、作弊行为，愿意接受《上海大学学生考试违纪、作弊行为界定及处分规定》的纪律处分。

应试人 _____ 应试人学号 _____ 应试人所在院系 _____

题号	一(20)	二(30)	三(25)	四(25)
得分				

得分	

一、判断题（每小题 2 分，共 20 分）

1. 引用在声明时必须对其初始化，以绑定某个已经存在的变量（或对象），在该引用的生命期内，该绑定不能被更改。 (√)
2. 指针变量在定义时必须对其初始化，以锁定某个已经存在的目标变量（或对象），在该指针变量的生命期内，该指向不能被更改。 (×)
3. 值返回的函数（如： double sqrt(double); ）的调用表达式（如：sqrt(2.0) ）代表一个无名的临时变量（或对象），一般不将其用作左值。 (√)
4. 引用返回的函数，可以返回该函数中 值传递 的形参变量（或对象）。 (×)
5. 任何类都有构造函数、复制构造函数、析构函数、赋值运算符函数。 (√)
6. 有静态数据成员的类，一般地应该考虑为其设计复制构造函数、析构函数。 (√)
7. 将用于输出的插入运算符函数 operator << 设计成友元函数的 根本原因 是因为进行输出操作时需要访问对象的内部数据成员。 (×)
8. 在 C++程序中，操作符 new 的功能与 calloc 函数的功能完全一样。 (×)
9. 创建一个 C++字符串对象（如： string str; ），则 sizeof (str) 的值等于 str.length() 的值。其中成员函数 length 为返回字符串的长度。 (×)
10. 基类的私有数据成员在派生类中是存在的，但不可直接访问，需要用从基类继承下来的函数访问。 (√)

得分 | | 二、填空题（每空 2 分，共 30 分）如下设计了结点类 Node、链表类 LinkList，并将链表类作为结点类的友类，请根据运行结果，完成程序。

```
#include <iostream>
using namespace std;
class ① LinkList ;                    // 提前声明
class Node
{
public :
    Node( int x=0) : data(x) {}
    ② friend class LinkList;
private :
    int data;
    Node ③ *next ;
};

class LinkList
{
private :
    Node *head;
public :
    LinkList( int n=0, int *array=NULL)
    {
        Node *p;
        head = NULL;
        for ( int i = ④ n-1 ; i>=0; i--)
        {
            p = new Node;
            p->data = (array!=NULL)?array[i]:0;
            p->next = head;
            ⑤ head = p ;
        }
    }
```

```
LinkList(       const    LinkList &link)
{
      head =    ⑥    NULL      ;
      * this    = link;
}
virtual       ~LinkList()
{
      FreeList();
}
⑦    LinkList &          operator    =( const    LinkList &link)
{
      if  ( ⑧    &link == this                  )
            return     *  this    ;
      FreeList();
      Node *p, *q;
      for  (p=link.head;          ⑨    p!=NULL     ; p=p->next)
      {
            if  (head==NULL)
                  head = q =       new  Node(p->data);
            else
            {
                  q->next =       new  Node(p->data);
                  q = q->next;
            }
      }
      ⑩    q->next = NULL           ;
      return     *  this    ;
}
```

```
    void    FreeList()
    {
        Node *p;
        while   ( ?     head != NULL          )
        {
            p = head;
            ?     head = head->next           ;
            delete     p;
        }
    }
    void    ShowList(ostream &out)          ?     const
    {
        out << "head";
        for   (Node *p=head; p!=NULL; p=p->next)
            out << " -> " << p->data;
        out << " -> NULL";
    }
};
ostream &      operator     <<(ostream &out,          const    LinkList &link)
{
    link.ShowList        ( ?     out    );
    return      ?     out       ;
}

int    main()
{
    int    n, a[] = {3, 7, 2, 1};
    n =    sizeof    (a)/   sizeof    (*a);
    LinkList linkA(n, a), linkB(linkA), linkC;
    cout << linkA << '\n' << linkB << '\n' << linkC << endl;
    linkC = linkA;
    cout << linkC << endl;
    return     0;
}
```

运行结果

```
head -> 3 -> 7 -> 2 -> 1 -> NULL
head -> 3 -> 7 -> 2 -> 1 -> NULL
head -> NULL
head -> 3 -> 7 -> 2 -> 1 -> NULL
```

得分 | |

三、阅读程序写出运行结果（每行 1 分，共 25 分）

1. （7 分）有关构造与析构的顺序

```
#include <iostream>
using namespace std;

class Test
{
public:
    Test( int a=0, int b=0): x(a), y(b) {}
    ~Test()
    {
        if (x==y)
            cout << "  数据成员的值相同，都等于  " << x << endl;
        else
            cout << " 数据成员的值不同，分别为  " << x << ", " << y << endl;
    }
    friend ostream & operator <<(ostream &out, const Test &t)
    {
        out << "(" << t.x << ", " << t.y << ")";
        return out;
    }
private:
    int x, y;
};

int main()
{
    Test *p, t1;
    p = new Test(10);
    Test t2(2, 3);
    cout << *p << '\n'
         << t1 << '\n'
         << t2 << endl;
    delete p;
    cout << "  退出程序，返回操作系统  " << endl;
    return 0;
}
```

运行结果 (1)

(10, 0)

(0, 0)

(2, 3)

数据成员的值不同，分别为 10, 0

退出程序，返回操作系统

数据成员的值不同，分别为 2, 3

数据成员的值相同，都等于 0

2. 以下两小题所涉及的类设计，头文件如下。

```
// test.h
#include    <iostream>
#include    <cmath>
using    namespace   std;
class    BASE
{
public    :
    BASE( double    x=0,    double    y=0): _x(x), _y(y) {}
    virtual    void    Show(ostream &out)    const    = 0;
protected    :
    double    _x, _y;
};

ostream    &   operator    <<(ostream &out,    const    BASE &x)
{
    x.Show(out);
    return    out;
}

class    Complex :    public    BASE
{
public    :
    Complex(   double    x=0,    double    y=0): BASE(x, y)
    {
    }
    void    Show(ostream &out)    const
    {
        if   (_x!=0)
        {
            out << _x;
            if   (_y>0)            out << '+' << _y << 'i';
            else    if   (_y<0)    out << '-' << -_y << 'i';
        }
        else
        {
            if   (_y!=0)
                cout << _y << 'i';
            else
                cout << _x;
        }
    }
```

```
friend Complex operator +( const Complex &a, const Complex &b)
{
    Complex c;
    c._x = a._x + b._x;
    c._y = a._y + b._y;
    return c;
}
friend Complex operator *( const Complex &a, const Complex &b)
{
    Complex c;
    c._x = a._x*b._x - a._y*b._y;
    c._y = a._x*b._y + a._y*b._x;
    return c;
  }
  double abs()
  {
    return sqrt(_x*_x + _y*_y);
  }
};

class Point : public BASE
{
public :
  Point( double x=0, double y=0): BASE(x, y)
  {
  }
  void Show(ostream &out) const
  {
    out << '(' << _x << ", " << _y << ')';
  }
  friend Point operator +( const Point &a, const Point &b)
  {
    Point c;
    c._x = a._x + b._x;
    c._y = a._y + b._y;
    return c;
  }
};
```

2.1 （10 分）测试程序

```
#include      "test.h"
int     main()
{
        Complex x(1,2), y(3,4), z1(0,1), z2(10);
        cout << x << '\n' << y << '\n'
                << z1 << '\n' << z2 << endl;
        cout << "y.abs(): " << y.abs() << endl;
        z1 = x+y;
        z2 = x*y;
        cout << z1 << '\n' << z2 << endl;

        Point a(1, 2), b(3, 4);
        cout << a << '\n' << b << endl;
        cout << a+b << endl;
        return      0;
}
```

运行结果 (2.1)

```
1+2i
3+4i
1i
10
y.abs()      : 5
4+6i
-5+10i
(1, 2)
(3, 4)
(4, 6)
```

2.2 （8 分）测试程序

```
#include      "test.h"
int     main()
{
        Complex x(1, 2), y, z;
        y = 5+x;
        z = 5*x;
        cout << x <<'\n'<< y <<'\n'<< z << endl;
        cout << x+x << endl;

        Point a(1, 2), b, c;
        b = a + 1;
        c = 1 + a;
        cout << a <<'\n'<< b <<'\n'<< c << endl;
        a = a + a;
        cout << a+a << endl;
        return       0;
}
```

运行结果 (2.2)

```
1+2i
6+2i
5+10i
2+4i
(1, 2)
(2, 2)
(2, 2)
(4, 8)
```

得分	

四、完成如下类的设计（25 分）在 GCC 编译系统中，unsigned long long 数据类型使整型数的取值范围得到扩展（ $0 \sim 2^{64}-1$，即 0~18 446 744 073 709 551 615。为了进一步扩展非负整数的取值范围设计了如下的类。该类数据可精确计算至 2^{123}，可处理 36~37 位非负十进制整数。 请在类的声明体外实现 5 个尚未定义的成员函数或友元函数。最后写出程序的运行结果（每个函数定义 4 分，运行结果 5 分）。

```
// LLINT.h                                          头文件
#ifndef     LLINT_H
#define     LLINT_H
#include     <iostream>
using     namespace   std;

class    LLINT
{
public    :
     LLINT( unsigned     long    long   x0=0,     unsigned     long    long   x1=0);
                                    //   第一参数为低位
     LLINT( const     char   *str);
     LLINT &     operator     ++();
     LLINT    operator     ++(int);
     friend     LLINT     operator     +( const   LLINT &x1,        const    LLINT &x2);
     LLINT &     operator     +=( const    LLINT &x);
     friend     ostream &        operator     <<(ostream &out,          const    LLINT &x);
     friend     LLINT atoLLINT(         const     char    *str);
     friend     istream &        operator     >>(istream &in, LLINT &x);
     friend      bool     operator     > (   const    LLINT &x1,        const    LLINT &x2);
     friend      bool     operator     >=(  const    LLINT &x1,        const    LLINT &x2);
     friend      bool     operator     < (   const    LLINT &x1,        const    LLINT &x2);
     friend      bool     operator     <=(  const    LLINT &x1,        const    LLINT &x2);
     friend      bool     operator     ==(  const    LLINT &x1,        const    LLINT &x2);
     friend      bool     operator     !=(   const    LLINT &x1,        const    LLINT &x2);
protected      :
     static       const     unsigned      long     long    BBILLION;
     unsigned       long     long    a1, a0;
     //a1*1 000 000 000 000 000 000 + a0                    可表示  36~37   位十进制非负整数
};
#endif

// LLINT.cpp                                        源程序文件
#include      "LLINT.h"
#include      <iomanip>

const    unsigned     long    long    LLINT::BBILLION       =  10000000000000000000ULL;
                        //   静态常量数据成员的定义及初始化（    10^18)
LLINT::LLINT(        unsigned long long            x0,      unsigned long long            x1)
{                       //   构造函数
      unsigned long long            x = x0/BBILLION;
      a0 = x0 % BBILLION;
      a1 = x1 + x;
}
```

```
LLINT::LLINT(       const     char   *str)          //   转换构造函数（从   C-字符串转换）
{
      * this    = atoLLINT(str);                    //   直接利用成员函数实现转换构造
}
LLINT LLINT::       operator    ++( int   )         //   后增量运算符函数
{
      LLINT temp(*      this   );
      ++(*  this   );
      return     temp;
}

LLINT    operator    +( const    LLINT &x1,       const    LLINT &x2)
{
      LLINT s;
      unsigned long long            c = x1.a0 + x2.a0;
      s.a0 = c % LLINT::BBILLION;
      s.a1 = x1.a1 + x2.a1 + c/LLINT::BBILLION;
      return     s;
}

ostream &       operator    <<(ostream &out,          const    LLINT &x)
{
      if   (x.a1!=0)
            out   <<  x.a1   <<  setfill('0')<<setw(18)<<                x.a0   <<  setfill('          ');
      else
            out << x.a0;
      return     out;
}

istream &       operator    >>(istream &in, LLINT &x)
{
      char   str[200];
      in >> str;
      x = atoLLINT(str);
      return     in;
}

bool    operator    >( const    LLINT &x1,       const    LLINT &x2)
{
      if   (x1.a1 > x2.a1)
            return       true   ;
      else     if   (x1.a1 == x2.a1)
            return     x1.a0 > x2.a0;
      else
            return       false    ;
}
```

```
bool operator       <( const    LLINT &x1,       const    LLINT &x2)
{
      if   (x1.a1 < x2.a1)
            return true;
      else if       (x1.a1 == x2.a1)
            return      x1.a0 < x2.a0;
      else
            return false          ;
}
bool operator       <=( const    LLINT &x1,       const    LLINT &x2)
{
      if   (x1.a1 < x2.a1)
            return true;
      else if       (x1.a1 == x2.a1)
            return      x1.a0 <= x2.a0;
      else
            return false          ;
}
LLINT atoLLINT(       const      char    *str)
{
      LLINT x;
      int    i, j=0, n;
      unsigned long long              p0=1, p1=1;
      for   (n=0; str[n]; n++)
            ;
      if   (n==0)      return      x;
      for   (i=n-1; i>=0; i--)
      {
            if   ('0'<=str[i] && str[i]<='9')
            {
                  if   (j<18)
                  {
                        x.a0 += p0*(str[i]-'0');
                        p0 *= 10;
                  }
                  else if       (j<36)
                  {
                        x.a1 += p1*(str[i]-'0');
                        p1 *= 10;
                  }
                  j++;
            }
      }
      return      x;
}
```

```
// LLINT_test.cpp                                测试程序
#include      "LLINT.h"
int     main()
{
        LLINT x("888 777 666 555 444 333 234 567 890 987 654 321  "),
                        y(100), z;
        cout << x << '\n' << y << '\n' << z << endl;
        z = "999 999 999 999 999 999";
        cout << z++ << endl;
        cout << z << endl;
        return      0;
}
```

运行结果

```
888777666555444333234567890987654321
100
0
9999999999999999
1000000000000000000
```

```
//      请在类模板体外定义成员函数及友元函数。      【提示】可充分利用已有的函数。
//      ①  (4 分)前增量运算符函数重载
LLINT & LLINT::         operator      ++()
{
        a0++;
        if(a0==BBILLION)
        {
                a0 = 0;
                a1++;
        }
        return      *  this    ;
}
```

```
//    ②  (4 分)加赋值运算符函数重载
LLINT & LLINT::        operator     +=( const    LLINT &x)
{
      * this     = *    this     + x;
      return      *  this    ;
}



//    ③  (4 分)关系运算符（大于或等于）函数重载
bool     operator     >=( const    LLINT &x1,        const    LLINT &x2)
{
      if   (x1.a1 > x2.a1)
            return       true    ;
      else     if   (x1.a1 == x2.a1)
            return      x1.a0 >= x2.a0;
      else
            return       false    ;
}



//    ④  (4 分)关系运算符（等于）函数重载
bool     operator     ==( const    LLINT &x1,        const    LLINT &x2)
{
      return      (x1.a1==x2.a1) && (x1.a0==x2.a0);
}



//    ⑤  (4 分)关系运算符（不等于）函数重载
bool     operator     !=(   const    LLINT &x1,        const    LLINT &x2)
{
      return      (x1.a1!=x2.a1) || (x1.a0!=x2.a0);
}
```